

Workload-Aware Deployment Pattern Selection for Cloud-Native Commerce Systems

Koteswararao Nallabothu
Lead Software Engineer
Lowe's
Charlotte, NC, USA
Nallabothukoteswar@gmail.com

Naresh Alapati
Principal Cloud Architect
Walmart
Bentonville, AR, USA
narencachy@gmail.com

Abstract—Commerce services require concurrent request serving while some background effects demand a single effective writer. Rolling, blue-green, and canary releases control traffic exposure but do not by themselves stop a paused former worker from committing a stale effect. Highlander separates serving capacity, effect discipline, and release selection, and binds a database lease holder to a monotonically increasing epoch in one acquisition transaction. A participating sink checks ownership and epoch within the effect transaction; unique operation keys and source-version checks address different duplicate and ordering hazards. The paper specifies the selection procedure and safety assumptions, then evaluates eight mechanism combinations using scheduled writes against PostgreSQL 16.15. Across 343 scheduled runs, the ordered workload comprised 30 trials per condition and 100 delayed attempts per trial. A source-version check admitted no version regressions; the co-located fence admitted no epoch regressions. Without a fence, a leased condition admitted 2,991 of 3,000 delayed attempts as epoch regressions; a restricted remote sink admitted nine of 3,000 older-epoch attempts after successor grant but before its first remote write. In 30 lost-ack retry schedules, response caching returned 165 stable replays with no observed mismatches. The results characterize the declared SQL schedules and transaction boundaries, not deployment throughput, outage frequency, or production cost.

Index Terms—fencing, idempotency, leader election, Kubernetes, deployment, reliability

I. INTRODUCTION

An inventory worker can resume after a pause and publish feed version 7 after another worker has published version 8. Unique operation keys do not prevent this rollback: the publications have different identities. A sink conditional on a usable source version does prevent it without leadership. Conversely, a unique key can suppress duplicate loyalty accrual without a leader. These examples expose two correctness requirements that must be evaluated separately. Serving capacity is independent of both. A checkout API can require many active replicas and a provider idempotency key for payment capture at the same time.

Rolling, blue-green and canary methods control how versions receive traffic [1], [2], [3]; they do not establish exclusive effect ownership. We use *Highlander* as an operational policy name, not a Kubernetes feature or a new consensus algorithm. It denotes one effective writer per conflict key at a sink that checks epochs when order follows ownership or the sink

cannot compare source versions. Old and new processes may execute simultaneously, but a stale epoch must not change protected state.

Our contributions are (i) a three-lane decision procedure combining capacity, availability, effect semantics and release risk; (ii) an ownership protocol whose acquisition atomically binds holder and epoch, together with a sink safety argument; and (iii) an ablation protocol that distinguishes duplicates, version regressions, epoch regressions, late accepts and rejected work against a conditional-version baseline. Sections II–IV give context, selection and protocol. Section V evaluates the protocol and deployment trade-offs. Sections VI–VII discuss applicability and future work.

II. BACKGROUND AND RELATED WORK

Leases bound ownership in time [4]; Chubby [5], ZooKeeper [6] and Raft [7] provide coordination and replicated-state foundations. They do not make a third-party payment or notification service transactional. Fencing tokens [8], retry-safe effects [9] and compensating transactions [10] address different boundaries. A token orders writers; an operation key identifies a repeated request; a saga compensates committed work. None is interchangeable with the others.

Borg, Omega and later Borg traces demonstrate heterogeneous cluster scheduling [11], [12], [13]. MillWheel and Photon offer stronger processing semantics within their own state and output models [14], [15]; those guarantees do not automatically cover arbitrary external APIs. Continuous-experimentation research [16] and canary analysis [17] motivate staged exposure and version-specific metrics. Kubernetes supplies rolling updates [1], Jobs [18], readiness [19], Leases [20] and autoscaling [21]; Argo adds progressive delivery controls [22]. We do not claim to invent leases, fencing or canary releases. Table I positions the systematization.

III. SYSTEM MODEL AND PATTERN SELECTION

Let λ_{peak} be peak requests/s, μ the measured safe throughput of one replica at a declared request mix and latency objective, and $h \in [0, 1]$ the reserve fraction. Approximately balanced serving requires

$$n_{\text{req}} = \left\lceil \frac{\lambda_{\text{peak}}}{\mu(1-h)} \right\rceil. \quad (1)$$

TABLE I
SCOPE RELATIVE TO PRIOR WORK

Work	Primary scope	Selection contribution
Leases / locks	Ownership	No topology/release rule
Keys / sagas	Retry/compensation	No capacity rule
Clusters	Placement/resources	No sink policy
Canary	Exposure/metrics	No epoch guarantee
This work	Three decisions	Rule and ablations

Input: $\text{serveRequests}, \lambda_{\text{peak}}, \mu, h, R, \text{RTO}, D, \rho, \text{SLO}, E, f, T$.

Output: (serving topology, effect discipline, release).

- 1) If serveRequests and $(\mu$ or R unknown), return `MEASURE_FIRST` for the component.
- 2) **Serving:** If serveRequests , compute (1)–(2). If $n_{\text{req}} > 1$ or (2) fails, select active-active; otherwise single-active is eligible after risk acceptance. If not serving, set serving topology to N/A.
- 3) **Effect:** Independently classify E : none $\rightarrow$ no effect guard; keyable/local transaction $\rightarrow$ unique operation key; source-versioned state $\rightarrow$ conditional version write; authority-ordered state $\rightarrow$ per-key lease and epoch fence; external non-idempotent effect $\rightarrow$ provider key, outbox and reconciliation.
- 4) Combine disciplines if effects overlap. If $D = 0$ and required sink/provider protection is unavailable, block the effect path pending redesign; if $D > 0$, record and monitor an explicit duplicate policy. Ownership may also reduce contention and rejected preparation work.
- 5) **Release:** Independently, high behavioral ρ selects canary analysis gates; high rollback urgency selects blue-green with compatible state; otherwise select rolling with availability controls.

Algorithm 1: Deployment Pattern Selection

This planning condition is not a latency guarantee: dependencies, skew and queueing can invalidate the assumed μ . For rare, non-overlapping failures, let f be expected failures per period T and R mean takeover time. A single-active service passes the availability screen only if

$$fR \leq (1 - \text{SLO})T, \quad R \leq \text{RTO}. \quad (2)$$

Tail-sensitive objectives require a percentile or correlated-failure analysis beyond this expectation. A standby may reduce recovery time but contributes no serving throughput.

Let D be tolerated duplicate effects per period, ρ the predeclared release-risk category, and E the effect semantics. Algorithm 1 evaluates every component in three lanes. A composite application receives both a serving plan and a worker/effect plan; effect handling never substitutes for capacity sizing. Unknown serving measurements return `MEASURE_FIRST`, avoiding arithmetic on a missing μ . A batch-only component has no requests/s sizing branch. Figure 1 summarizes the lanes.

For illustration, $\lambda_{\text{peak}} = 600$, $\mu = 120$ requests/s and $h = 0.20$ give $\lceil 600/96 \rceil = 7$. Six replicas provide 576 requests/s after reserve and fail the condition; seven provide 672 requests/s, or 72 requests/s beyond the assumed peak. These are declared inputs and arithmetic, not observations. The availability screen remains symbolic until f and R are measured. Table II gives composite outcomes.

IV. HIGHLANDER ARCHITECTURE AND SAFETY

A. Effect boundary and atomic ownership

Figure 2 separates requests from asynchronous work. For order-confirmation email, an outbox durably records intent,

TABLE II
WORKLOAD-TO-PATTERN MAPPING

Workload	Serving plan	Effect discipline
Checkout API	Sized active-active	Provider key; reconcile capture
Internal UI	(1)–(2) determine eligibility	Keyed local writes where needed
Ordered feed job	N/A	Source version check; lease + fence for owner-defined order or costly contention
Export API + job	Active-active API	Per-export key; bounded workers

while a provider key or reconciliation addresses an ambiguous acknowledgment. Payment capture needs provider idempotency. An inventory feed with usable source versions can use a conditional version write; a search-index alias whose order is determined by its current publisher needs epoch fencing. Loyalty accrual often admits a unique business operation key. A system with several sinks needs protection at each sink; one database transaction cannot confer exactly-once behavior on an uncooperative external API.

Our PostgreSQL implementation binds ownership and epoch: an owner row $(k, \text{holder}, e, \text{expires_at})$ is changed by a conditional update on the database clock. Acquisition assigns holder, increasing epoch and expiry in the *same* transaction. Renewal requires matching holder and epoch and an unexpired row. Each process incarnation has a UUID. A sink-fenced write locks that row and atomically checks authority with its ledger/state update.

We call this the *lease-then-epoch race*: W1 acquires a Kubernetes Lease, pauses before requesting a separate epoch, and resumes after W2 has acquired a lease and token. Giving W1 a later token now legitimizes a zombie. Our protocol has no separate token-allocation operation: W1’s delayed conditional acquisition is rejected while W2 owns an unexpired row. A pause after successful acquisition leaves W1 with its old epoch. Figure 4(b) illustrates this fix. A Kubernetes-only alternative must bind a holder change and epoch to the same compare-and-swap update, preserve monotonicity across object recreation, and use unique incarnation identities; it is not the protocol evaluated here.

B. State transitions and safety property

Table III and Fig. 3 describe local control states. “Active” is a client’s belief, not proof that other processes stopped. Shutdown and uncertain renewal enter Draining. Crashes from Candidate, Active or Draining enter Failed; recovery returns to Standby only after durable-state reconciliation.

For the combined discipline, let $e_{\text{current}}(k)$ be the durable current epoch and $\mathcal{K}(k)$ the committed operation keys. The participating sink enforces

$$\text{accept}(k, e, op) \Rightarrow e \geq e_{\text{current}}(k) \wedge op \notin \mathcal{K}(k). \quad (3)$$

In the co-located fenced conditions, the implementation strengthens the epoch check to equality with the current holder

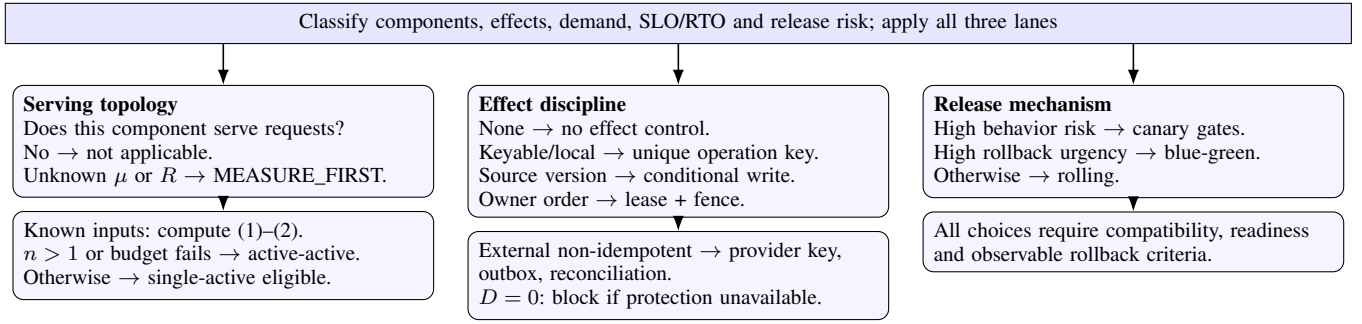


Fig. 1. Three independent selection lanes. A checkout API can receive active-active serving, provider idempotency controls and a canary release simultaneously. The measurement exit applies only to request-serving components.

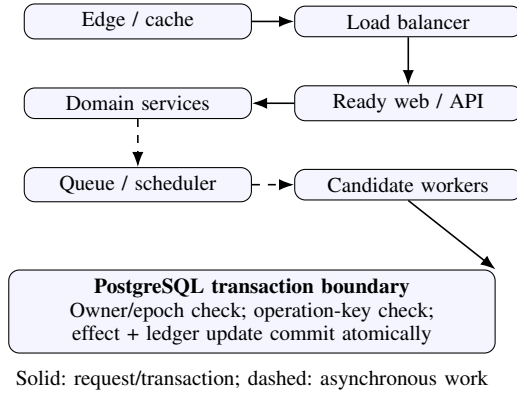


Fig. 2. Reference architecture. The asynchronous branch is dashed. In the testbed, ownership validation, ledger insertion and effect mutation share a PostgreSQL transaction boundary.

TABLE III
WORKER TRANSITIONS

From	Event [guard] / action	To
Standby	work [enabled] / acquire	Candidate
Candidate	grant [atomic CAS wins] / begin	Active
Candidate	denied [valid owner] / back off	Standby
Active	renew fails [uncertain] / stop admission	Draining
Active	shutdown [requested] / checkpoint	Draining
Draining	settled [durable checkpoint] / stop	Standby
Candidate, Active, Draining	crash [any] / no further action	Failed
Failed	recover [reconciled] / rejoin	Standby

and valid expiry; tokens are never accepted merely because a client presents a larger number. The effect and ledger update commit atomically. Monotonic epoch issuance and a serialized check-and-write reject a former epoch after its successor is installed. Atomic uniqueness rejects an already committed operation key even within the same epoch. This proves neither cross-sink atomicity nor exactly-once external effects, and

does not forbid an earlier valid write before the successor acquires ownership. In fence-only C2f, only the epoch clause is enforced; in unique-only C1u, only the operation-key clause is enforced. C1v instead checks a monotone data version at the sink without acquiring ownership.

C. Time and failure assumptions

A client-side safety deadline is $t_{\text{renew}} + L - \delta$: effects must finish before that deadline. For Kubernetes Lease clients, expiry is judged from local observations, not an API-server expiry clock. Here δ conservatively includes clock-rate drift between candidates, observation delay and write-to-commit time [23]. No finite client-only margin proves safety under unbounded pauses. In our PostgreSQL implementation the database clock decides expiry and the sink transaction decides authority; client stopping is an optimization, not its safety foundation.

Measured from the last successful renewal, acquisition and activation satisfy $R_{\text{renew}} \leq L + P + S$ under bounded coordination and scheduling. From a crash Δ after renewal, time to the first accepted write instead satisfies

$$R_{\text{accept}} \leq \max(0, L - \Delta) + P + S + W. \quad (4)$$

P covers detection/polling and coordinator response; S covers activation and work preparation through the first successful submission; W covers sink acceptance. A drain term G is added only when a required serialized drain extends beyond expiry, not when it already occurred inside L . Unbounded partition or sink delay invalidates a finite bound. Expand/contract schema changes and backward-compatible checkpoints remain necessary: neither a traffic switch nor a ready Pod guarantees a safe worker-state rollback [19].

V. EVALUATION

A. Testbed and scope

The completed controlled experiment executes PostgreSQL 16.15 sink functions and SQL attempt logs on a GitHub Actions PostgreSQL service. The restricted remote sink uses a separate schema and role without access to ownership; this is a permission boundary in one database instance, not an independent networked service. A separate planned campaign uses

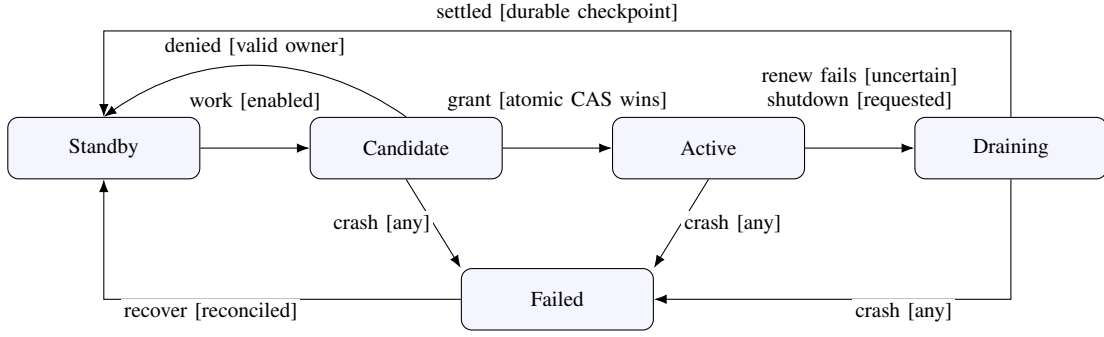
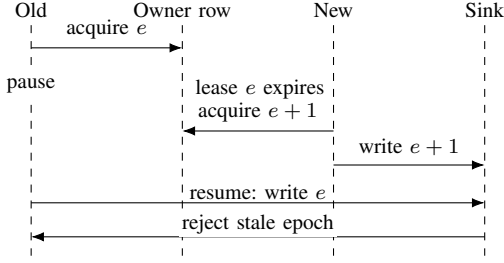
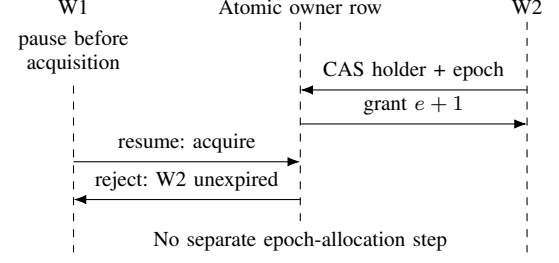


Fig. 3. Local states with event [guard] labels. The curved denial edge returns Candidate to Standby.



(a) Stale effect after lease expiry



(b) Atomic acquisition rejects a delayed contender

Fig. 4. Two distinct pause hazards. (a) A sink rejects an old epoch after a successor’s publication. (b) Binding holder and epoch in one conditional transaction prevents the separate-allocation race.

TABLE IV

MECHANISM ABLATIONS. LEASE IS A POSTGRESQL OWNER-ROW LEASE; CO-LOCATED FENCE CHECKS HOLDER, EPOCH AND EXPIRY; C3R CHECKS ITS SINK EPOCH.

Condition	Lease	Fence	Version	Unique	Replay
C1	–	–	–	–	–
C1u	–	–	–	Yes	–
C1v	–	–	Yes	–	–
C2	Yes	–	–	–	–
C2f	Yes	Yes	–	–	–
C3	Yes	Yes	–	Yes	–
C3r	Yes	Remote	–	Yes	–
C4	Yes	Yes	–	Yes	Yes

a two-node kind cluster, three Python candidates and a CPU-limited HTTP service. The eight mechanism combinations are listed in Table IV.

B. Methodology and hypotheses

The planned kind campaign would compare the eight mechanisms at $b = 10$ and $d = 1$ s with 30 trials per applicable workload and 1,000 items per trial; a separate sweep varies $b \in \{1, 10, 100\}$ and $d \in \{0, 1, 5\}$ s. The duplicate workload reuses operation keys. In the ordered workload, the source assigns comparable versions independently of any lease; each publication uses a distinct operation key. A version regression is an accepted lower source version, and an epoch regression is an accepted lower authority epoch. Both use recorded pre-write state. Epoch regression applies only where a lease epoch exists.

In the planned fault campaign, a worker prepares a batch, self-stops with SIGSTOP, and resumes with SIGCONT after a successor grant or first write. The runner verifies process state and a stopped heartbeat [24]; batch size controls exposure. Hypotheses distinguish repeated-key suppression (C1u/C3), source-version monotonicity (C1v), authority fencing (C2 versus C2f/C3), and stable replay (C4). C3r advances its feed-wide remote fence on the successor’s first remote write, leaving a grant-to-write window. This sink boundary explains why a prior-epoch accept can occur even when no version regression occurs.

The completed lost-ack series retries a current-owner operation key after 50 ms under a predeclared 5% acknowledgment-loss input. C2f may repeat the effect, C3 rejects an ambiguous repeat, and C4 replays its cached response. The planned cluster campaign would separately report rejected attempts, preparation CPU, invalid trials and released at-risk submissions. For zero observed safety violations, it would use a one-sided 95% Clopper–Pearson upper bound; continuous trial summaries would use 10,000 percentile-bootstrap resamples with seed 2026. These inferential procedures are not applied to the scripted data.

Planned takeover trials vary $L \in \{5, 10, 15, 30\}$ s with SIGKILL and no lease release, bracket the kill, and compare first accepted writes with (4). Serving tests would calibrate μ at a p99 objective, compare $n_{\text{req}} - 1$ with n_{req} , and measure release intervals for rolling, blue-green and client-weighted canary. Replica-seconds proxy allocation rather than dollars; Table V defines the endpoints.

TABLE V
EVALUATION ENDPOINTS

Metric	Definition	Raw source
Duplicates	Accepted repeated key	Sink attempts
Version regressions	Accepted source version < stored version; all conditions	Prior sink state
Epoch regressions	Accepted epoch < maximum applied epoch; lease conditions	Prior sink state
Late accepts	Old epoch after grant, before new sink write	Grant/accept logs
Wasted work	Rejections; preparation CPU	Sink attempts
Takeover	Kill bracket to first effect	Fault/accept logs
p99; violations	Latency; SLO failures	HTTP requests
Release cost	Extra replica-seconds	Readiness logs

C. Results

1) *Scheduled PostgreSQL interleavings*: The completed study executes the PostgreSQL owner-row, sink, and remote-sink SQL functions under a controlled event schedule; the earlier SQLite rule-model study provides a comparison, not the primary result. Every ordered trial starts with a v1 publication on each key. Across 30 paired schedules of 100 keys per condition, the successor publishes v3 first for each shuffled key with a declared input probability of 70%; the delayed former worker attempts v2. All conditions use the same trial seeds. A v2 accepted after v3 is a version regression, whereas an older epoch accepted after any successor write is an authority regression. These measures have different denominators and must not be conflated. In particular, the feed-wide authority metric may flag an old-epoch write even if that particular item has not received v3.

Table VI is generated from the PostgreSQL trial CSV. The unguarded C1 and C2 conditions each admit 2071 version regressions. C2 also admits 2991 epoch regressions. C1v rejects late lower-version attempts without acquiring a lease. The co-located fence rejects all delayed former-epoch attempts after grant, so C2f, C3, and C4 have zero observed epoch regressions under this schedule. These are observed SQL outcomes in the specified trial cells; no independence-based failure-rate bound follows from repeated keys or chosen ordering.

C3r stores one remote sink epoch for the whole feed, advancing it on the successor’s first remote effect. Its 9 accepted former-epoch effects occur after the successor grant and before that first remote write. The old effects precede v3 on their individual keys, explaining why this cell has zero version regressions despite a grant-to-write authority gap. In the after-grant sweep, the successor first remote write is scheduled at 2 s and the former batch resumes at $d \in \{0, 1, 5\}$ s. Each cell is a single deterministic schedule of 100 keys. The remote sink accepts all 100 older-epoch attempts at $d = 0$ and $d = 1$ s and rejects all at $d = 5$ s; the co-located fence rejects all former-epoch attempts at each delay. Figure 5 plots only

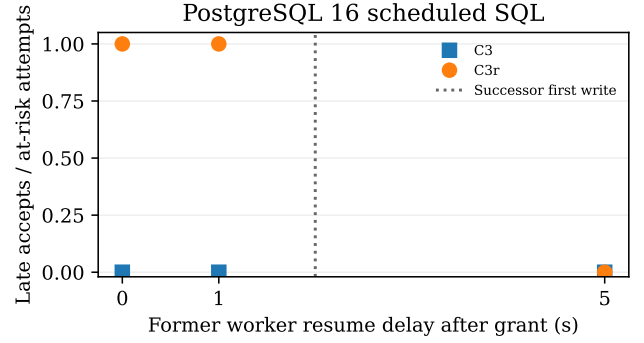


Fig. 5. PostgreSQL after-grant late-accept fraction from scheduled SQL trial rows. Markers are tested delays; the dotted line is the scheduled first successor write, not an observed latency.

the tested delays; intermediate delays were not measured.

The duplicate pause series has 1 run per applicable condition and repeats the same business operation keys after takeover. C1 and C2 accept those duplicate effects; C1u enforces uniqueness. The co-located fence rejects delayed repeats in C2f/C3/C4, while C3r uses its unique key and/or remote epoch. This single run does not estimate a population duplicate rate. The independent lost-ack series exercises retries by the *current* owner: 30 paired schedules use 100 initial calls each, 5% input acknowledgment-loss probability, and 50 ms before retry. Of 165 realized retries per condition, C2f accepts duplicate effects, C3 rejects duplicates without resolving the caller’s response, and C4 returns stable cached responses with zero observed mismatches (Table VII).

2) *Active handover and acquisition races*: The former writer starts submitting before takeover, while a separate thread expires the setup lease, acquires epoch 2 and begins successor writes. Each of 5 repetitions includes 30 runs per condition with 40 keys; randomized microdelays and host scheduling vary the overlap. The former’s setup lease is expired by an administrative update after its initial batch commits, and the successor then acquires epoch 2 through `own.acquire`. Table VIII counts legitimate pre-grant accepts separately from former writes committed after grant; the latter are obtained from the SQL `late_accepts` view. The co-located fences have no post-grant former accepts; C3r has 436 across the five repetitions, ranging from 77 to 92 per repetition. Separate two-session races on 1,000 fresh keys each produce one winner at epoch 1; 1,000 expired-key takeover races each produce one winner at epoch 2, incrementing the prior epoch by 1. These finite races test one PostgreSQL instance, not independent network partitions.

The PostgreSQL 16.15 CI run reports 343 raw schedules, 43 summary rows, zero mismatched cell metrics relative to the SQLite model, and 53 passing tests (run 36291161498). The equality of selected aggregates validates agreement under shared scripted inputs; it does not demonstrate independent production replication or cluster behavior. Neither deployment velocity, preparation CPU, replica-seconds, takeover latency,

TABLE VI

SCRIPTED POSTGRESQL 16 INTERLEAVING OUTCOMES. COUNTS REFLECT THE SPECIFIED SCHEDULE AND SINK RULES, NOT FAILURE-RATE ESTIMATES.

Condition	n_D	n_O	Duplicates	Version reg.	Epoch reg.	Late	Rejected _O
C1	1	30	100	2,071	N/A	N/A	0
C1u	1	30	0	2,071	N/A	N/A	0
C1v	N/A	30	N/A	0	N/A	N/A	2,071
C2	1	30	100	2,071	2,991	N/A	0
C2f	1	30	0	0	0	N/A	3,000
C3	1	30	0	0	0	0	3,000
C3r	1	30	0	0	0	9	2,991
C4	1	30	0	0	0	N/A	3,000

n_D/n_O : scheduled duplicate/ordered runs; each run has 100 keys. Version and epoch regressions are separate outcomes on the ordered workload. Late means an accepted former-epoch write after grant and before the successor's first remote write. Rejected_O: delayed ordered attempts rejected by a guard. N/A: inapplicable or not measured. CPU and provider cost are not measured.

TABLE VII

SCRIPTED LOST-ACK RETRIES BY THE CURRENT OWNER. OUTCOME COUNTS FOLLOW THE CHOSEN LOSS DRAWS; ZERO RESPONSE MISMATCHES WERE OBSERVED.

Condition	Retry calls	Dup. accepted	Ambig. rejected	Stable replay
C2f	165	165	0	0
C3	165	0	165	0
C4	165	0	0	165

TABLE VIII

ACTIVE-HANDOVER RACES ON POSTGRESQL. ONLY SQL COMMITS AFTER EPOCH-2 GRANT COUNT AS LATE ACCEPTS.

Condition	Runs	Pre-grant accepted	Late / post-grant
C2f	150	1,494	0/4,138
C3	150	1,487	0/4,131
C4	150	1,490	0/4,125
C3r	150	1,866	436/4,134

p99 latency, nor provider effects were measured, so no economic or downtime reduction is inferred.

D. Threats to validity and artifact availability

The planned kind cluster shares one host; CPU contention and host faults would correlate otherwise separate Pods. The scheduled study cannot characterize unconstrained concurrency; the threaded check covers limited real concurrency on one PostgreSQL instance. Neither measures cluster faults, external service behavior, or operational cost. Its restricted remote sink is a separate schema and role within the same PostgreSQL instance, with no network partition. Synthetic effects omit provider behavior, cache skew and session affinity. SIGSTOP captures a defined delayed batch rather than every possible partition. PostgreSQL's transaction boundary is stronger than many external sinks. Cluster clock changes can affect expiry; bound interpretation requires clock and delay assumptions as well as a point estimate.

Artifact Availability: The paper, raw CSVs, SQL, and generators are archived at <https://github.com/nallabothukoteswar-wq/highlander-harness>, tag `paper-v9.1`; the earlier `paper-v9` snapshot is retained. The v9.1 handover generator was introduced at commit

`a85c5ffaafa31bdb1c001b747c9ac837964a0440`.

The measured handover run is archived at <https://github.com/nallabothukoteswar-wq/highlander-harness/actions/runs/36334206522>. The kind campaign runner remains under development; reported counts come from the included CSVs and scripts. No archival DOI is available.

VI. DISCUSSION AND LIMITATIONS

Idempotency keys solve repeatable, keyable effects. A conditional version check blocks version regressions when source versions are comparable. Epoch fencing blocks epoch authority regressions for lease conditions when a lower epoch follows a higher one; a remote epoch fence can still admit an older epoch after a successor grant but before its first remote write. These metrics must be analyzed separately. Ownership can reduce contention and wasted work but does not prove that only one process is executing. An ordered workload must also define order within one epoch; our test worker publishes sequentially. If current-epoch operations can be reordered in transit, an additional version check is required. A fencing result must not be generalized to ordering that the sink never checks.

A payment or email provider remains outside the local transaction: an outbox preserves intent but cannot settle an unknown external outcome alone. Per-key ownership avoids a global bottleneck for independent feeds. Interactive APIs retain active-active capacity sizing even when their effect paths are serialized. A low-traffic internal interface is single-active only when capacity and availability constraints both pass. During coordinator loss, stopping effects favors safety at the cost of progress. Blue-green rollback needs compatible state; canaries need representative traffic and version-tagged metrics. PodDisruptionBudgets restrict voluntary disruptions, not node loss [25].

The analytical results establish a decision decomposition and a conditional sink safety argument. The scheduled PostgreSQL outcomes expose distinct version, epoch, and remote late-accept cases under their chosen schedules; they do not establish production speedup, cost savings, or failure-rate reduction. PostgreSQL protocol tests for uniqueness, fencing, atomic acquisition, response replay and restricted remote-sink

access passed in the cited CI run; the scheduled study does not exercise a Kubernetes failover campaign.

VII. CONCLUSION AND FUTURE WORK

Workload-aware deployment requires separate decisions for serving topology, effect discipline and release mechanism. Atomic ownership binds a holder to its fencing epoch, while atomic operation-key handling prevents repeat effects. The distinction is necessary to evaluate Highlander as a single-effective-writer policy without attributing uniqueness guarantees to leadership. Future work should assess production traces, multi-zone coordinator faults, reordered current-epoch publications and real provider idempotency contracts. The reported sink observations cover scheduled SQL writes and bounded single-instance thread races; cluster and production claims require the planned kind campaign.

REFERENCES

- [1] Kubernetes Authors, “Deployments,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/> (accessed Sep. 26, 2026).
- [2] M. Fowler, “Blue Green Deployment,” 2010. [Online]. Available: <https://martinfowler.com/bliki/BlueGreenDeployment.html> (accessed Sep. 26, 2026).
- [3] M. Fowler, “Canary Release,” 2014. [Online]. Available: <https://martinfowler.com/bliki/CanaryRelease.html> (accessed Sep. 26, 2026).
- [4] C. G. Gray and D. R. Cheriton, “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency,” in *Proc. ACM SOSP*, 1989. <https://doi.org/10.1145/74850.74870>
- [5] M. Burrows, “The Chubby Lock Service for Loosely-Coupled Distributed Systems,” in *Proc. USENIX OSDI*, 2006. <https://www.usenix.org/conference/osdi-06/chubby-lock-service-loosely-coupled-distributed-systems>
- [6] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-free Coordination for Internet-scale Systems,” in *Proc. USENIX ATC*, 2010. <https://www.usenix.org/conference/usenix-atc-10/zookeeper-r-wait-free-coordination-internet-scale-systems>
- [7] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *Proc. USENIX ATC*, 2014. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [8] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017. <https://dataintensive.net/>
- [9] P. Helland, “Idempotence Is Not a Medical Condition,” *ACM Queue*, 2012. <https://queue.acm.org/detail.cfm?id=2187821>
- [10] H. Garcia-Molina and K. Salem, “Sagas,” in *Proc. ACM SIGMOD*, 1987. <https://doi.org/10.1145/38713.38742>
- [11] A. Verma *et al.*, “Large-scale cluster management at Google with Borg,” in *Proc. ACM EuroSys*, 2015. <https://research.google/pubs/large-scale-cluster-management-at-google-with-borg/>
- [12] M. Schwarzkopf, A. Konwinski, M. Abd-El-Malek, and J. Wilkes, “Omega: flexible, scalable schedulers for large compute clusters,” in *Proc. ACM EuroSys*, 2013. <https://research.google/pubs/omega-flexible-scalable-schedulers-for-large-compute-clusters/>
- [13] M. Tirmazi *et al.*, “Borg: the Next Generation,” in *Proc. ACM EuroSys*, 2020. <https://doi.org/10.1145/3342195.3387517>
- [14] T. Akidau *et al.*, “MillWheel: Fault-Tolerant Stream Processing at Internet Scale,” *Proc. VLDB Endowment*, 2013. <https://research.google/pubs/millwheel-fault-tolerant-stream-processing-at-internet-scale/>
- [15] R. Ananthanarayanan *et al.*, “Photon: Fault-tolerant and Scalable Joining of Continuous Data Streams,” in *Proc. ACM SIGMOD*, 2013. <https://research.google/pubs/photon-fault-tolerant-and-scalable-joining-of-continuous-data-streams/>
- [16] G. Schermann, J. Cito, P. Leitner, U. Zdun, and C. H. Gall, “We’re doing it live: A multi-method empirical study on continuous experimentation,” *Information and Software Technology*, vol. 99, 2018. <https://eprints.cs.univie.ac.at/5953/>
- [17] Š. Davidović and B. Beyer, “Canary Analysis Service,” *ACM Queue*, vol. 16, no. 1, 2018. <https://queue.acm.org/detail.cfm?id=3194655>
- [18] Kubernetes Authors, “Jobs,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/controllers/job/> (accessed Sep. 26, 2026).
- [19] Kubernetes Authors, “Liveness, Readiness, and Startup Probes,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/pods/probes/> (accessed Sep. 26, 2026).
- [20] Kubernetes Authors, “Leases,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/leases/> (accessed Sep. 26, 2026).
- [21] Kubernetes Authors, “Horizontal Pod Autoscaling,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/> (accessed Sep. 26, 2026).
- [22] Argo Project, “Canary Deployment Strategy,” *Argo Rollouts Documentation*. [Online]. Available: <https://argo-rollouts.readthedocs.io/en/stable/features/canary/> (accessed Sep. 26, 2026).
- [23] Kubernetes Authors, “leaderelection package,” client-go source documentation. [Online]. Available: <https://pkg.go.dev/k8s.io/client-go/tools/leaderelection> (accessed Sep. 26, 2026).
- [24] Linux man-pages project, “pid_namespaces(7),” *Linux manual page*. [Online]. Available: https://man7.org/linux/man-pages/man7/pid_namespaces.7.html (accessed Sep. 26, 2026).
- [25] Kubernetes Authors, “Specifying a Disruption Budget for your Application,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/tasks/run-application/configure-pdb/> (accessed Sep. 26, 2026).